TRABAJO FIN DE GRADO · SMR

Nexus-Core

Guía educativa y técnica

Monitorización, administración y seguridad de infraestructura con agentes remotos. Arquitectura, datos, redes, seguridad y laboratorios para clase.

Saúl Moreno · Málaga · Octubre de 2026
Documento educativo · Versión 1.0

Cómo usar esta guía

Esta guía está pensada para que un profesor pueda explicar Nexus-Core en clase y para que el alumnado entienda **qué hace**, **cómo está construido** y, sobre todo, **por qué** se tomaron cada una de las decisiones. No es un manual de usuario: es un recorrido por la arquitectura, los datos, la seguridad y las redes de una plataforma real, con ejercicios seguros de laboratorio.

Los ejemplos usan nombres reservados para documentación (`nexus.example.com`, `192.0.2.10`) y marcadores como `<TOKEN>`. Ninguna credencial real aparece en este documento.

- Cómo usar esta guía
- 1. Qué es Nexus-Core
  - 1.1 El problema que resuelve
  - 1.2 Objetivos de diseño
- 2. Arquitectura general
  - 2.1 Por qué hub + agentes
  - 2.2 El protocolo agente-hub
  - 2.3 Qué ocurre cuando el hub está apagado
- 3. Funciones del panel
  - 3.1 Alertas: del dato a la decisión
  - 3.2 Monitores y página de estado
  - 3.3 IA con human-in-the-loop
  - 3.4 Instalación de servicios (apt)
  - 3.5 Terminal local y por agente
  - 3.6 Auditoría de seguridad (solo lectura)
- 4. Datos y persistencia
  - 4.1 Muestreo y agregación
  - 4.2 Copias de seguridad
- 5. Multi-host: una flota, un panel
  - 5.1 Alta de un host
  - 5.2 Selección de host
  - 5.3 Límites del agente
- 6. Seguridad
  - 6.1 Capas de protección
  - 6.2 Autenticación y 2FA
  - 6.3 Sesiones, CSRF y cookies
  - 6.4 Superficie de ataque y privilegios
  - 6.5 Secretos y configuración
  - 6.6 Despliegue en Internet
- 7. Redes
  - 7.1 Escaneo de la red local

    - 7.2 Agentes y NAT
    - 7.3 Docker y puertos
    - 7.4 Cabeceras de proxy y HTTPS
- 8. Instalación y operación
    - 8.1 Hub nativo (recomendado)
    - 8.2 Primer arranque
    - 8.3 Agente en otra máquina
    - 8.4 Backups y actualización
    - 8.5 Diagnóstico rápido
    - 8.6 Cliente Android nativo
    - 8.7 Edición privada self-hosted y licencias
- 9. Laboratorios para el aula
    - Laboratorio 1 — Arquitectura
    - Laboratorio 2 — Monitorizar una VM
    - Laboratorio 3 — Alerta de disco
    - Laboratorio 4 — Servicio caído
    - Laboratorio 5 — Inventario de red
    - Laboratorio 6 — 2FA y recuperación
    - Laboratorio 7 — Backup y restauración
    - Laboratorio 8 — Publicación segura
- 10. Preguntas de repaso
- 11. Glosario
- 12. Apéndices
    - 12.1 Endpoints principales
    - 12.2 Herramientas del agente
    - 12.3 Estructura del proyecto
    - 12.4 Buenas prácticas de operación

1. Qué es Nexus-Core

Nexus-Core convierte un servidor Linux en un **centro de control** para vigilar y administrar otras máquinas. Desde una única página web se puede ver la telemetría del sistema, gestionar contenedores Docker, revisar servicios, lanzar un escaneo de la red local, consultar puertos, actualizaciones y discos, recibir alertas y ejecutar acciones con confirmación.

La plataforma nace como Trabajo Fin de Grado de SMR, pero está construida como un producto real: un *hub* central, agentes que se instalan en otras máquinas y una interfaz que funciona en local y a través de Internet.

> **La idea en una frase:** un panel único, ligero y comprensible, que convierte datos sueltos (CPU, RAM, servicios, contenedores, red) en decisiones y en acciones auditables.

1.1 El problema que resuelve

En un aula o en una pequeña infraestructura, la información está repartida: métricas en una herramienta, servicios en otra, contenedores en la terminal, la red en un escaneo manual y las alertas en el correo. Nexus-Core reúne esas señales en un solo lugar y añade tres cosas que suelen faltar en un TFG:

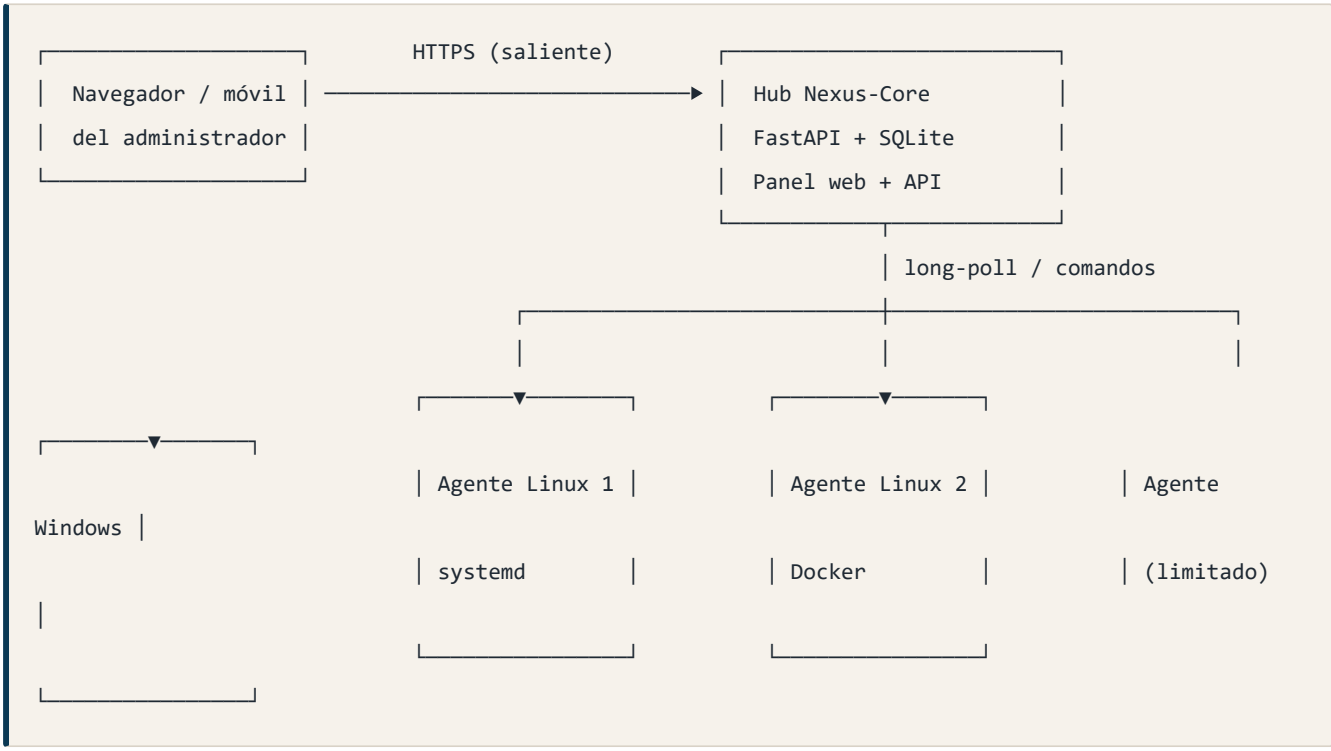
1. **Contexto**: no solo muestra un número, explica si está bien, regular o mal.
2. **Acción controlada**: permite actuar (reiniciar un servicio, desplegar un contenedor) con lista blanca, confirmación y auditoría.
3. **Trazabilidad**: cada acción queda registrada; el administrador puede reconstruir qué pasó.

1.2 Objetivos de diseño

Objetivo	Decisión
Funcionar en una máquina modesta (Raspberry Pi, VM)	Backend Python ligero, SQLite, frontend sin compilación
No depender de la nube	Todo el núcleo funciona en local; Internet es opcional
Poder vigilar máquinas detrás de NAT	El agente inicia la conexión **hacia fuera**; no se abren puertos
Ser entendible para un alumno	Código modular, nombres explícitos, documentación y auditoría
No convertir la IA en un riesgo	La IA **propone**; una persona confirma acciones sensibles
Poder desplegarse de forma segura	TLS en el borde, cookies Secure, 2FA, rate limiting y backups

2. Arquitectura general

Nexus-Core tiene tres piezas:



- **Hub**: servidor central. Ejecuta el panel, la API, la base de datos, el motor de alertas, los monitores y el despachador de comandos.
- **Agentes**: procesos instalados en cada máquina vigilada. Recogen métricas y ejecutan herramientas bajo demanda. **Solo abren conexiones salientes** hacia el hub.
- **Navegador**: consume el panel. No necesita instalar nada.

2.1 Por qué hub + agentes

Un panel que "entra" en cada máquina necesitaría abrir puertos, configurar cortafuegos y gestionar credenciales en cada host. El modelo de agente invierte la dirección: cada máquina abre una conexión saliente al hub. Esto permite funcionar detrás de NAT, en redes de aula o en un servidor doméstico sin IP fija. Es el mismo patrón que usan herramientas comerciales de monitorización.

2.2 El protocolo agente-hub

1. **Alta (`enroll`)**: el agente presenta un token de enrolamiento y recibe un token propio para ese host. El token de enrolamiento es un secreto de alta; el token del host identifica a la máquina en cada petición.
2. **Latido (`heartbeat`)**: cada pocos segundos el agente informa de que sigue vivo y envía su inventario (contenedores, servicios, versión).
3. **Long-poll de comandos**: el agente mantiene una petición abierta esperando órdenes. El hub responde en cuanto hay una herramienta que ejecutar, sin que el agente tenga que sondear agresivamente.
4. **Resultado**: el agente devuelve `done` o `error`. El hub guarda el comando y su resultado.

> **Por qué long-poll y no WebSocket:** el long-poll es más simple de operar (HTTP normal, proxies, TLS) y suficiente para una herramienta de administración. Un WebSocket añadiría estado y reconexiones que este proyecto no necesita.

2.3 Qué ocurre cuando el hub está apagado

Los agentes siguen funcionando: reintentan con una espera que crece de 2 a 30 segundos. No se pierde la configuración ni los servicios de la máquina. Cuando el hub vuelve, los agentes reconectan solos. Las métricas del periodo de caída no se backfillean (el agente no guarda un búfer histórico); es una limitación consciente y una posible mejora futura.

---

3. Funciones del panel

Pestaña	Qué enseña	Por qué importa
Resumen	Sensores (CPU, RAM, swap, disco, carga, temperatura), veredicto global, alarmas y uso de recursos	Es la vista de un vistazo; el color y el texto evitan tener que interpretar números
Contenedores	Lista Docker, estado, puertos, logs, consumo, despliegue y borrado con lista blanca	Docker es el corazón de muchos despliegues; se gestiona sin terminal
Servicios	Inventario systemd, estado, logs del journal y acciones start/stop/restart/reload	Permite diagnosticar y recuperar un servicio desde el navegador
Red	Interfaces, IP/CIDR, MAC, MTU, puerta de enlace, escaneo de hosts y configuración guiada	La red es la base de todo; el escaneo ayuda a inventariar el aula
Sistema	Puertos en escucha, actualizaciones apt y estado SMART	Tres preguntas clásicas de soporte: qué escucha, qué falta actualizar y cómo está el disco
Alertas	Reglas por umbral, duración, histéresis, acknowledgment, snooze y notificaciones	Convierte la telemetría en avisos accionables, no en ruido
Monitores	HTTP, TCP y SSL con latidos, uptime y página de estado	Vigila servicios desde el punto de vista del usuario
Histórico	Series temporales con rangos y exportación CSV	Permite explicar tendencias y justificar decisiones
Auditoría	Registro encadenado de accesos y acciones	Sin trazabilidad no hay administración responsable
Auditoría de seguridad	Sistema, identidad, archivos, servicios, Docker, red, firewall y app; solo lectura	Diagnóstico accionable sin tocar el host
Seguridad	Tokens de API, sesiones, 2FA TOTP y códigos de recuperación	Protege el panel y las credenciales
Asistente	Chat con IA y propuestas de acciones pendientes de confirmar	Acelera el diagnóstico sin dar autonomía peligrosa a la IA
Hosts	Inventario de la flota, alta de agentes, estado y comandos	Es la puerta al modelo multi-máquina

3.1 Alertas: del dato a la decisión

Una alerta no es “CPU al 90 %”. Una regla define un umbral, una duración mínima para evitar falsos positivos, un umbral de recuperación (histéresis) y unos canales de notificación. El estado pasa por `ok → aviso → crítico → recuperada`; el administrador puede reconocerla o silenciarla durante un mantenimiento.

> **Por qué la histéresis:** sin ella, un valor que oscila alrededor del umbral generaría decenas de eventos. Con ella, la alerta se abre cuando se cruza el umbral y se cierra solo cuando baja del umbral de recuperación.

3.2 Monitores y página de estado

Los monitores comprueban HTTP, TCP y certificados SSL. Su resultado alimenta una página pública de estado y un endpoint de métricas para Prometheus. Es una separación deliberada: la telemetría del host no es lo mismo que la disponibilidad de un servicio.

3.3 IA con human-in-the-loop

El asistente usa *function calling*: el modelo no ejecuta código libre, solo puede pedir herramientas declaradas. Las acciones sensibles se convierten en **propuestas pendientes** que el administrador confirma o rechaza. El objetivo es que la IA ayude a diagnosticar sin quitar el control.

3.4 Instalación de servicios (apt)

El panel incluye un catálogo cerrado de servicios instalables con apt: Nginx, Samba, dnsmasq, Docker, Fail2ban y Redis. No acepta nombres de paquete libres: la clave se valida contra `/etc/nexus/installable.json` y un helper root-owned con allowlist. Cada instalación exige confirmación y queda registrada en Auditoría.

- En el host local del hub, el backend llama al helper mediante una regla sudoers exacta.
- En un agente Linux, la orden viaja por el despachador y el agente llama a su propio helper.
- Docker añade el usuario del servicio al grupo `docker`; hay que reiniciar `nexus` o `nexus-agent` para que el proceso lo vea.
- Para que apt pueda escribir, las unidades relajan `ProtectSystem` y `NoNewPrivileges`; la aplicación y el agente siguen ejecutándose como usuarios no privilegiados y solo el helper tiene root.

> **Por qué allowlist y no un campo de paquete:** instalar paquetes es una operación de root. Un campo libre convertiría el panel en una shell root; el catálogo cerrado mantiene la superficie de ataque pequeña y auditable.

3.5 Terminal local y por agente

La pesta?a **Terminal** abre una shell real dentro del navegador con xterm.js y un WebSocket autenticado. Hay dos transportes:

- **Hub local**: una PTY de PowerShell en Windows o de la shell del servicio en Linux. Se activa con `TERMINAL_LOCAL_SHELL=true`.
- **Agente remoto**: el agente Nexus 1.1.0 abre un WebSocket saliente contra el hub y expone una PTY en el host. El navegador nunca inicia una conexi?n hacia el servidor: funciona detr?s de NAT, sin abrir puertos y sin SSH.

- **SSH**: queda como fallback opcional para instalaciones antiguas, pero no es necesario para la terminal por agente.

La sesi?n exige cookie de administrador, token CSRF en el subprotocolo `csrf.<token>`, mismo origen y cierre por inactividad. El host local ejecuta la shell con el usuario del servicio; los agentes Linux/Kali usan `pty` y Windows usa ConPTY/pywinpty. No hay una shell root impl?cita.

El desplegable **Servidor** muestra el transporte (`local` , `agente` o `SSH`). Los agentes 1.0.0 aparecen deshabilitados con el motivo ?actualiza el agente? hasta que se instala la versi?n 1.1.0.

Variables relevantes:

```
TERMINAL_LOCAL_SHELL=true
TERMINAL_LOCAL_SHELL_COMMAND=powershell.exe -NoLogo -NoProfile
TERMINAL_AGENT_ENABLED=true
TERMINAL_IDLE_SECONDS=900
```

El canal del agente usa `/api/agent/terminal` ; el navegador usa `/api/terminal/ws?host=ID` . Ambos est?n autenticados y auditados.

3.6 Auditoría de seguridad (solo lectura)

La pestaña **Auditoría** incorpora un auditor propio inspirado en Lynis, pero con una vuelta de tuerca: cruza sistema, identidad, archivos, servicios, Docker, red, firewall y la propia aplicación, y devuelve cada hallazgo con **severidad, evidencia y recomendación**. La diferencia clave es que **no modifica nada**: no escribe configuración, no reinicia servicios, no instala paquetes y no toca permisos. Solo lee y ejecuta comandos de consulta.

- **Sistema:** plataforma, actualizaciones pendientes, reinicio, unattended-upgrades y NTP.
- **Identidad:** UID 0, contraseñas vacías, SSH (root, contraseña, puerto, X11) y sudoers.
- **Archivos:** permisos de `nexus.env` , clave SSH, base de datos, token de setup y auditoría; world-writable y SUID/SGID.
- **Servicios y Docker:** unidades fallidas, usuario del servicio, contenedores privilegiados, red/PID host, root, `:latest` , socket montado y permisos del socket.
- **Red y firewall:** puertos en escucha, exposición en `0.0.0.0` , Docker API, bases de datos y estado de ufw/nftables o de los perfiles de Windows. Si `nmap` está instalado, se añade un escaneo activo de loopback (`-sT -Pn --top-ports 100 127.0.0.1`), sin scripts ni exploits; si no, se omite. ZAP no se ejecuta desde el hub: se plantea como escaneo externo.
- **Aplicación:** 2FA, cookie Secure, `ALLOWED_HOSTS` , CORS, caducidad de sesión, cadena de auditoría, tokens de API, flota, backup y límite de enrolamiento.
- **Windows:** Defender, firewall, SMBv1, RDP y último hotfix.

La puntuación parte de 100 y resta pesos por hallazgo (25 crítico, 15 alto, 7 medio, 3 bajo; un aviso resta la mitad). Los checks omitidos no penalizan, pero se muestran con su motivo. En Linux se ejecutan los checks de sistema, SSH, systemd y permisos POSIX; en Windows se sustituyen por Defender, SMBv1, RDP y firewall, y los de Linux se marcan como omitidos.

El endpoint `GET /api/audit/security` devuelve el informe cacheado cinco minutos; `POST /api/audit/security/run` fuerza una nueva pasada. Ambos exigen autenticación y el POST exige CSRF. El informe incluye `read_only=true`, host, plataforma, duración, puntuación, contadores y hallazgos; la interfaz permite filtrar por severidad y exportar el JSON. La auditoría no escribe en la cadena de auditoría ni en ningún otro sitio.

Baseline y comparación. El botón **Guardar baseline** crea `security_audit_baseline.json` (0600) en el directorio de datos con la puntuación y los hallazgos actuales; **Borrar baseline** lo elimina. Con un baseline guardado, el panel muestra una tarjeta de comparación: nuevos, regresiones, resueltos, los que siguen fallando y el delta de puntuación. Endpoints: `POST` y `DELETE` `/api/audit/security/baseline`. Guardar o borrar el baseline es la **única escritura** y es una acción explícita; el resto del auditor sigue siendo de solo lectura.

---

4. Datos y persistencia

Nexus-Core usa **SQLite** para simplificar el despliegue: un solo archivo, sin servidor de base de datos, fácil de copiar y restaurar. El esquema evoluciona con migraciones versionadas.

Tabla	Contenido
`metric_samples`	Muestras de CPU, RAM, swap, disco, carga y temperatura, por host
`metric_rollups`	Agregados por minuto (1m), 5 minutos (5m) y hora (1h)
`hosts`	Inventario de la flota (local y agentes), estado y token
`host_state`	Último inventario del agente (contenedores, servicios, versión)
`commands`	Órdenes enviadas a un agente, estado, resultado y error
`alert_rules` / `alert_events`	Reglas, umbrales, estados y acknowledgments
`notification_channels`	Telegram, webhook o correo
`maintenance_windows`	Ventanas de silencio planificadas
`monitors`	Comprobaciones HTTP/TCP/SSL y su historial

4.1 Muestreo y agregación

- El colector muestrea cada **5 segundos**.
- Se calculan agregados por minuto (1m), 5 minutos (5m) y hora (1h) para que el histórico no crezca sin límite.
- La retención y los rollups permiten mantener meses de tendencia con un peso pequeño.
- El histórico se consulta por host y por rango; el CSV exporta exactamente lo que se ve.

4.2 Copias de seguridad

El script `deploy/production/backup-nexus.py` usa la **API de backup de SQLite**: crea un snapshot consistente aunque el servicio esté en marcha, comprueba `PRAGMA integrity_check`, incluye la configuración y rota las copias. Un backup que no se ha probado a restaurar no es un backup.

---

5. Multi-host: una flota, un panel

5.1 Alta de un host

1. El administrador abre **Hosts** y copia el **token de enrolamiento**.
2. En la máquina destino instala el agente con ese token y un nombre.
3. El agente se enrola, recibe su token propio y empieza a latir.
4. El hub lo muestra como un host más y permite elegirlo en el selector.

> **Por qué hay dos tokens:** el de enrolamiento sirve para dar de alta máquinas nuevas; el token del host identifica a una máquina concreta y puede revocarse sin afectar a las demás.

5.2 Selección de host

El selector global decide qué host alimenta el **Resumen** y el **Histórico**. Las pestañas de **Contenedores, Servicios, Red y Sistema** ejecutan la operación en el host seleccionado:

- Si es el host local del hub, se usan los módulos locales.
- Si es un agente, la orden viaja por el despachador (`POST /api/hosts/{id}/command`) y el agente la ejecuta con una **allowlist** de herramientas.

5.3 Límites del agente

El agente puede ejecutar: métricas, Docker (listar, logs, stats, desplegar, eliminar, prune), servicios (listar, detalle, logs, acciones), red (interfaces, escaneo) y sistema (puertos, actualizaciones, SMART). No ejecuta código arbitrario. Un host Windows no tiene systemd ni Docker accesible con la misma semántica: la UI muestra un aviso claro en lugar de fingir.

---

6. Seguridad

La seguridad no es una pestaña: es una propiedad del diseño. Estas son las capas, de fuera hacia dentro.

6.1 Capas de protección

Capa	Medida	Por qué
Borde	TLS (Caddy o Cloudflare Tunnel)	Cifra el tráfico y evita credenciales en claro
Red	Firewall: solo SSH de entrada; el resto sale	El hub no necesita puertos abiertos hacia Internet
Aplicación	Cookies `Secure` + `HttpOnly` + `SameSite`, CSRF	Evita robo de sesión y peticiones cruzadas
Identidad	Contraseña fuerte + **2FA TOTP** + códigos de recuperación	Una contraseña filtrada no basta para entrar
Sesiones	Caducidad por inactividad y absoluta	Limita el daño de un equipo desatendido
Fuerza bruta	Rate limiting por usuario e IP	Frena ataques automatizados
Autorización	Lista blanca de herramientas e imágenes	El panel no es una shell remota
Privilegios	Servicio como usuario no root + helpers sudo acotados	Un fallo en la app no es root directo
Auditoría	Registro encadenado por hash	Detecta manipulación y permite reconstruir acciones
Datos	TOTP, IA y backups cifrados; claves fuera del directorio de datos	Una copia robada no debe revelar secretos
IA	Propuestas pendientes de confirmación	La IA no actúa sin una persona

6.2 Autenticación y 2FA

- La contraseña se almacena con un hash fuerte (Argon2id) y nunca se registra.
- El login tiene límite de intentos por usuario y por IP.
- **2FA TOTP**: Google Authenticator (o cualquier app compatible) genera un código de 6 dígitos cada 30 segundos. El servidor guarda el secreto y verifica una ventana corta.
- **Códigos de recuperación**: se muestran una sola vez, se guardan hasheados y sirven si se pierde el móvil. Deben guardarse fuera del teléfono.
- El panel exige el código en el **mismo formulario de login**, no en una pantalla opcional.

> **Por qué TOTP y no SMS:** TOTP no depende de la operadora, funciona sin cobertura y no expone el número de teléfono a terceros. Es el estándar de las cuentas profesionales.

6.3 Sesiones, CSRF y cookies

Las sesiones caducan por inactividad (30 minutos por defecto) y por tiempo absoluto (8 horas). Cada petición que modifica datos exige un token CSRF. La cookie de sesión se marca `Secure` cuando el panel se sirve por HTTPS; sin HTTPS, el navegador no la envía y el login no persiste, lo que evita un falso sentido de seguridad.

6.4 Superficie de ataque y privilegios

- La aplicación corre como usuario `nexus`, no como root.
- Las operaciones que requieren privilegios pasan por **helpers** concretos con reglas sudo validadas (`visudo`) y una lista blanca de unidades.
- El socket de Docker es potente: montarlo da un privilegio equivalente a root sobre el host. Por eso el despliegue con Docker se documenta como opción de laboratorio y se recomienda el servicio nativo en producción.
- El instalador apt solo acepta claves del catálogo; el helper root-owned valida la clave y ejecuta apt sin recibir comandos del panel. Las unidades relajan el sandbox para que apt escriba, pero la aplicación y el agente siguen como usuarios sin privilegios.
- El escaneo de red se limita a rangos privados y requiere autorización.
- Los tokens de API con ámbito `read` no pueden leer el token de enrolamiento.
- `/docs`, `/openapi.json` y Prometheus quedan cerrados por defecto; si se necesita Prometheus, se define `METRICS_TOKEN`.
- Los comentarios de Samba no aceptan CR/LF y el agente exige HTTPS; el hub limita el tamaño del cuerpo y los long-polls por host.
- El rate limiting usa `CF-Connecting-IP` cuando la petición llega desde loopback (Cloudflare), no una cabecera arbitraria del cliente.

6.5 Secretos y configuración

- `.env` y los tokens nunca se suben al repositorio; el repositorio público solo contiene ejemplos con marcadores.
- El token de enrolamiento se puede rotar; los hosts ya enrolados conservan su token propio.
- La configuración de producción vive en `/etc/nexus/nexus.env` con permisos 0600.
- TOTP, claves de IA y backups se cifran con Fernet; la clave maestra queda fuera de `NEXUS_DATA_DIR` y no se incluye en los backups. Los archivos siguen siendo 0600.

6.6 Despliegue en Internet

Dos opciones habituales:

1. **Cloudflare Tunnel**: `cloudflared` abre una conexión saliente y Cloudflare publica el panel con HTTPS. Un **Quick Tunnel** es gratis, no requiere cuenta y cambia de URL; sirve para pruebas. Un **Named Tunnel** con dominio es la opción estable. Cloudflare Access añadiría SSO, pero **no forma parte de esta versión**.

2. **VPS con Caddy**: el hub escucha en `127.0.0.1:8000`, Caddy termina TLS y añade HSTS y cabeceras de seguridad. El firewall solo permite SSH, 80 y 443.

En ambos casos el panel **no** debe escuchar en `0.0.0.0` cuando se publica. La unidad systemd trae `0.0.0.0:8000` para el primer arranque en LAN; `/etc/nexus/nexus.env` lo sobreescribe a `127.0.0.1:8000` y el proxy/túnel hace de puerta.

---

7. Redes

7.1 Escaneo de la red local

El escaneo busca hosts activos en una subred privada (RFC 1918). Usa `nmap` si está disponible y, si no, un barrido con tabla ARP como respaldo. El resultado muestra IP, estado, MAC y nombre. Solo debe usarse en redes propias o con autorización expresa.

> **Por qué se limita a rangos privados:** escanear redes ajenas es intrusión. La herramienta educativa debe enseñar el límite, no solo la técnica.

7.2 Agentes y NAT

El agente no necesita IP fija ni puertos abiertos: inicia una conexión HTTPS saliente y mantiene un long-poll. Funciona detrás de NAT, en una VM de aula o en una Raspberry Pi. El hub solo necesita ser alcanzable por el agente (por LAN o por el túnel).

7.3 Docker y puertos

El panel muestra el estado de los contenedores y los puertos publicados. El despliegue usa una lista blanca de imágenes y un límite de contenedores gestionados. Es una decisión de seguridad: no se permite desplegar cualquier imagen desde el panel.

7.4 Cabeceras de proxy y HTTPS

Cuando hay un proxy (Caddy, Cloudflare), uvicorn debe confiar en las cabeceras reenviadas (`--proxy-headers --forwarded-allow-ips=127.0.0.1`). Así la IP del cliente, el esquema HTTPS y las cookies Secure se comportan correctamente. Sin esa confianza, el panel vería todo el tráfico como localhost.

---

8. Instalación y operación

8.1 Hub nativo (recomendado)

```
sudo apt install -y git python3 python3-venv
git clone <repo> nexus-core
cd nexus-core
sudo bash deploy/install.sh
```

El instalador crea el usuario `nexus`, los directorios, el entorno virtual, los helpers, la unidad systemd y el token de primer arranque. El puerto depende del entorno:

Entorno	Escucha	Puerto	Acceso
Linux, primer arranque	`0.0.0.0`	`8000`	`http://IP:8000/login.html`
Linux, producción	`127.0.0.1`	`8000`	túnel o Caddy (`https://nexus.labsxz.com`)
App Windows	`127.0.0.1`	`8780`	ventana pywebview
Demostración	`127.0.0.1`	`8790`	solo capturas

El `0.0.0.0` es solo para el primer acceso en LAN; en producción, `/etc/nexus/nexus.env` fija `NEXUS_BIND_HOST=127.0.0.1`.

8.2 Primer arranque

```
sudo cat /var/lib/nexus/setup_token
```

Se abre el panel, se pega el token, se crea el administrador y se guarda la contraseña en un gestor. Después se activa el 2FA y se guardan los códigos de recuperación.

8.3 Agente en otra máquina

```
sudo bash deploy/agent/install-agent.sh \
  --hub https://nexus.example.com \
  --enroll-token '<TOKEN>' \
  --name "vm-aula-01"
```

El agente se instala como servicio systemd, se enrola y empieza a latir. No hay que abrir puertos en la máquina vigilada.

El instalador crea el venv, instala el subset de dependencias y ejecuta el enrolamiento desde `/opt/nexus-agent`; no lances `agent.py enroll` a mano desde otra ruta. Si el hub cambia de URL (por ejemplo, un Quick

Tunnel nuevo) o se reconstruye la base, el token y el `host_id` del host quedan obsoletos: vuelve a ejecutar el instalador con la URL y el token actuales. Se creará una entrada nueva en la flota; elimina la antigua si queda offline.

8.4 Backups y actualización

```
sudo systemctl enable --now nexus-backup.timer
sudo systemctl start nexus-backup.service
```

Actualizar el hub:

```
cd nexus-core && git pull
sudo bash deploy/update.sh
```

8.5 Diagnóstico rápido

Síntoma	Comprobación
El panel no carga	`systemctl status nexus` ; `curl http://127.0.0.1:8000/api/health`
Un host no aparece	`systemctl status nexus-agent` ; token, URL del hub (no un Quick Tunnel caducado) y salida HTTPS
El servicio está `enabled` pero no late	`journalctl -u nexus-agent -b` ; si ves `Name or service not known` y una URL `*.trycloudflare.com` , el túnel cambió: reenrola con la URL fija y un token nuevo
El host sale duplicado o con un ID viejo	El reenrolamiento crea una entrada nueva; borra la antigua offline en Hosts
El túnel da 530/502	El túnel no está conectado o el hub no escucha en localhost
La sesión no persiste	HTTPS y cookie Secure; usa la URL HTTPS, no HTTP
Docker no lista	El usuario del servicio no está en el grupo docker o el socket no existe

8.6 Cliente Android nativo

Además del panel web, Nexus-Core incluye una aplicación **Android nativa** en `android/` , escrita en **Kotlin + Jetpack Compose**. No usa WebView: toda la interfaz es nativa y consume la misma API HTTPS del hub.

- **14 secciones**: Resumen, Hosts, Contenedores, Servicios, Red, Alertas, Histórico, Monitores, Auditoría, Sistema, Seguridad, Terminal, Asistente y Ajustes.
- Login con **contraseña + TOTP** (Google Authenticator), cookie de sesión y CSRF.
- Selector global de host; la app no replica la base SQLite del hub, consulta la API.

- Permite gestionar hosts, contenedores, servicios, alertas, monitores, tokens, sesiones, auditoría, baseline, terminal local/por agente, asistente, cambio de IP, 2FA y la configuración de IA.
- La sesión se guarda cifrada con Android Keystore; no se incrustan tokens `write`.
- APK de debug instalable por sideload en Android 8.0+ (`minSdk 26`); ABIs `arm64-v8a`, `armeabi-v7a`, `x86` y `x86_64`. La release de Play Store a?n requiere firma de producci?n.
- Se compila abriendo `android/` en Android Studio (SDK 37) o con `android\gradlew.bat :app:assembleDebug`.

Límites actuales: las notificaciones en segundo plano usan WorkManager cada 15 minutos (no FCM), el cambio de IP solo aplica al hub local con netplan y la app no guarda una copia offline completa.

8.7 Edición privada self-hosted y licencias

La edición privada no usa el hub del vendedor. Cada comprador instala Nexus-Core en su propia máquina o servidor, con su dominio, su base de datos y su administrador. La APK solo contiene el cliente y pide la URL del hub en el primer arranque.

La licencia recomendada es **offline y firmada**: un archivo `nexus.license` emitido por el vendedor con identificador de comprador, caducidad, límite de dispositivos y firma Ed25519. La APK incluye únicamente la clave pública, por lo que no necesita contactar con un servidor central para validar la compra. La licencia no está vinculada a un dominio concreto.

Flujo de venta:

1. El comprador contacta por la web o el teléfono.
2. Recibe un enlace privado con `Nexus-Core.apk`, `nexus.license`, `SHA256.txt` e instrucciones.
3. Instala y configura su propio hub.
4. Introduce la URL HTTPS de su hub y activa la licencia.
5. Crea su administrador y activa su 2FA.

El APK debe compilarse en `release`, con keystore privado, R8 y sin `android:debuggable`. La instalación fuera de Google Play puede mostrar el aviso de “origen desconocido” o una advertencia de Play Protect; eso depende del canal, no del código. Para una distribución con confianza de tienda habría que usar una prueba cerrada de Play o Managed Google Play. Nunca se comparte el usuario `admin` entre compradores.

9. Laboratorios para el aula

> Todos los ejercicios se hacen en una red de laboratorio propia y con autorización. No se escanean redes ajenas ni se usan credenciales reales.

Laboratorio 1 — Arquitectura

Dibuja el flujo navegador → hub → agente. Identifica qué viaja por HTTPS y qué por long-poll. **Pregunta:** ¿por qué el agente no necesita abrir el puerto 22?

Laboratorio 2 — Monitorizar una VM

Instala un agente en una VM del aula y compárala con el host local. **Objetivo:** ver dos hosts en el selector y entender el concepto de flota.

Laboratorio 3 — Alerta de disco

Crea una regla de disco > 80 % con recuperación al 70 % y duración 5 minutos. Llena un archivo temporal hasta cruzar el umbral. **Observa:** cuándo se abre, cuándo se cierra y qué queda en Auditoría.

Laboratorio 4 — Servicio caído

Detén un servicio no crítico del laboratorio y comprueba el estado, los logs y la acción de reinicio. **Pregunta:** ¿qué privilegios necesita el panel y por qué no corre como root?

Laboratorio 5 — Inventario de red

Escanea la subred del laboratorio. Compara el resultado de nmap con el respaldo ARP. **Pregunta:** ¿qué diferencia hay entre un host que responde a ping y uno que bloquea ICMP?

Laboratorio 6 — 2FA y recuperación

Activa TOTP, cierra sesión y entra con el código. Después usa un código de recuperación. **Pregunta:** ¿por qué el servidor guarda los códigos de recuperación hasheados?

Laboratorio 7 — Backup y restauración

Ejecuta el script de backup, inspecciona el archivo y restaura en una copia de la base. **Objetivo:** comprobar integridad y entender que un backup sin restauración no es una garantía.

Laboratorio 8 — Publicación segura

Publica el panel con un túnel o un proxy TLS en el laboratorio. Verifica HSTS, cookies Secure y que el puerto 8000 no es accesible desde fuera. **Pregunta:** ¿qué **añadiría** Cloudflare Access? (No se implementó en esta versión.)

10. Preguntas de repaso

1. ¿Por qué el agente inicia la conexión y no el hub?
2. ¿Qué diferencia hay entre el token de enrolamiento y el token de un host?
3. ¿Por qué se usa histéresis en las alertas?
4. ¿Qué protege el CSRF y qué no protege?
5. ¿Por qué la cookie Secure impide iniciar sesión por HTTP?
6. ¿Qué riesgo tiene montar el socket de Docker en un contenedor?
7. ¿Por qué los rollups son importantes para el histórico?
8. ¿Qué significa que la IA esté en modo *human-in-the-loop*?
9. ¿Por qué el escaneo se limita a rangos privados?
10. ¿Qué diferencia un Quick Tunnel de un Named Tunnel?

---

11. Glosario

- **Hub**: servidor central que recibe telemetría y reparte órdenes.
- **Agente**: proceso en una máquina vigilada que envía métricas y ejecuta herramientas.
- **Long-poll**: petición HTTP que el servidor mantiene abierta hasta que hay una orden.
- **TOTP**: código temporal de 6 dígitos basado en tiempo (Google Authenticator).
- **CSRF**: ataque que fuerza a un navegador autenticado a enviar una petición no deseada.
- **TLS**: cifrado de la conexión (HTTPS).
- **Reverse proxy**: servidor que recibe el tráfico y lo reenvía al backend (Caddy, Nginx).
- **CIDR**: notación de subred, por ejemplo `192.0.2.0/24`.
- **NAT**: traducción de direcciones que permite salir a Internet sin IP pública.
- **Allowlist**: lista explícita de lo permitido (herramientas, imágenes, unidades).
- **Rollup**: agregado de datos para conservar histórico sin crecer indefinidamente.
- **SSRF**: forzar al servidor a hacer peticiones a destinos internos no previstos.
- **Human-in-the-loop**: la IA propone y una persona confirma.

---

12. Apéndices

12.1 Endpoints principales

Método	Ruta	Uso
GET	`/api/health`	Estado del hub
POST	`/api/auth/login`	Login (contraseña + TOTP si está activo)
GET	`/api/metrics`	Métricas del host local
GET	`/api/containers`	Contenedores del host local
GET	`/api/services`	Servicios systemd
GET	`/api/network/state`	Interfaces y estado de red
POST	`/api/network-scan`	Escaneo de subred
GET	`/api/system/ports`	Puertos en escucha
GET	`/api/system/updates`	Actualizaciones apt
GET	`/api/system/smart`	Estado SMART
GET	`/api/alerts/rules`	Reglas de alerta
GET	`/api/monitors`	Monitores HTTP/TCP/SSL
GET	`/api/hosts`	Inventario de la flota
POST	`/api/hosts/{id}/command`	Comando a un agente
POST	`/api/agent/enroll`	Alta de un agente
POST	`/api/agent/heartbeat`	Latido
GET	`/api/agent/commands`	Long-poll de comandos
POST	`/api/agent/commands/{id}/result`	Resultado
GET	`/api/metrics/prometheus`	Exportación Prometheus
GET	`/api/status/page`	Página pública de estado
WS	`/api/terminal/ws`	Terminal local/agente autenticada (xterm.js)
GET	`/api/terminal/targets`	Hosts conectados y disponibilidad de terminal

12.2 Herramientas del agente

- `metrics`

- `docker.list`, `docker.logs`, `docker.stats`, `docker.deploy`, `docker.remove`, `docker.prune`
- `services.list`, `services.detail`, `services.logs`, `services.action`
- `network.interfaces`, `network.scan`
- `system.ports`, `system.updates`, `system.smart`

12.3 Estructura del proyecto

```
backend/
  agent.py              agente remoto (enroll/run, allowlist, backoff)
  app/
    api/                rutas HTTP (auth, hosts, alerts, monitors, sistema, seguridad)
    core/               configuración, seguridad, flota, SQLite, TOTP, auditoría
    modules/            colector, Docker, systemd, red, SMART, FIM, diagnóstico local
    ai/                 proveedor, herramientas y acciones pendientes
frontend/               panel, login, ajustes y estilos
android/                app nativa Kotlin + Jetpack Compose
deploy/                 instalación, agentes, systemd, Caddy, producción y backups
docs/                   memoria del TFG y esta guía
```

> La herramienta remota `diagnostics` del agente **no está implementada**; el módulo `diagnostics.py` del hub genera el ZIP de diagnóstico local que descarga el panel.

12.4 Buenas prácticas de operación

- Cambia la contraseña inicial y activa 2FA antes de publicar el panel.
- Usa HTTPS; nunca publiques el puerto 8000 directamente.
- Guarda el token de enrolamiento como un secreto y rótalo si se filtra.
- Prueba la restauración de backups, no solo su creación.
- Mantén el servidor actualizado y con el firewall cerrado salvo lo imprescindible.
- Revisa la auditoría cuando ocurra algo raro.
- No escanees redes sin autorización.
- Trata las respuestas de la IA como propuestas, no como órdenes.

Nexus-Core es un proyecto educativo y profesional. Esta guía puede copiarse y adaptarse para clase citando el proyecto. Las marcas y nombres de terceros pertenecen a sus propietarios.